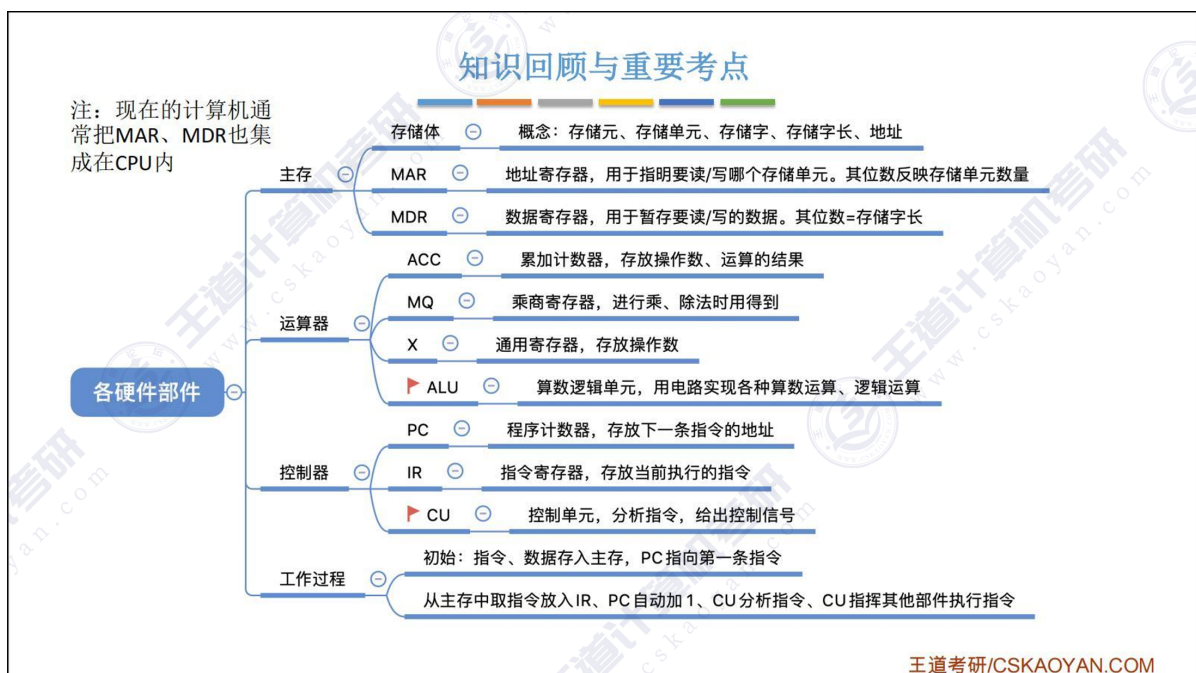


Computer Organization

第 1 章 计算机系统概述

硬件

1. 计算机系统 = 外设 + 主机
2. 字节 B , 比特 b : $1B = 8b$
 - 存储器容量单位: B
 - $1MB = 2^{10}KB$
 - 网络流量单位: bps , 这里的 b 指比特, $1000Mbps = \frac{1000}{8}MB/s = 125MB/s$
 - $1Mbps = 10^3kbps$
3. 相联存储器即可以按地址寻址也可以按内容寻址, 也称按内容寻址的存储器



软件

1. 软件和硬件的逻辑功能等价性: 同一个功能, 既可以用软件实现, 也可以用硬件实现
 - 软件成本低、性能低
 - 硬件成本高、性能高
2. 指令集体系结构 (Instruction Set Architecture ISA) 是硬件与软件之间的分界线
3. "兼容": 指计算机软件或硬件的通用性, 通常在同一系列不同型号的计算机间通用

计算机的性能指标

1. 运算速度指标: 吞吐量、响应时间、CPU 时钟周期、主频、CPI、IPS、MIPS、FLOPS
2. IPC: 每个时钟周期运行多少条指令

性能指标

指标	描述与公式	量纲	说明
机器字长	CPU 一次处理的二进制位数 ($8b$ 整数倍)	位 b	核心指标 , 决定寄存器/ALU 宽度, $1B = 8b$
数据通路带宽	数据总线单次并行传输位数	位 b	总线物理宽度 (如 64 位), $\leq$ 机器字长时可能成为瓶颈
主存容量	容量 $= 2^{MAR\text{位数}} B$	字节 B	MAR 位数决定寻址上限 , $1GB = 2^{30} B$
吞吐量	单位时间处理请求数	请求/秒	系统综合性能指标
响应时间	响应时间 = CPU 时间 + I/O 等待时间	秒 s	用户体验关键指标, $1s = 10^3 ms$
CPU 时钟周期	$T = \frac{1}{f}$ (f 为主频)	秒 s	CPU 最小时间单位 , $1s = 10^9 ns$
主频 (f)	$f = \frac{1}{T}$	赫兹 Hz	$1GHz = 10^9 Hz$, 非直接性能标准 (需结合 CPI)
CPI	$CPI = \frac{\text{总时钟周期数}}{\text{指令条数}}$	周期/指令	执行一条指令所需的平均时钟周期数, 指令效率核心指标 , CPI↓ 性能↑
IPS	$IPS = \frac{f}{CPI}$	指令/秒	每秒执行指令数, 直接反映指令执行速度
CPU 执行时间	时间 $= \frac{\text{总时钟周期数}}{f} = \frac{\text{指令数} \times CPI}{f}$	秒 s	运行一个程序所花费的 CPU 时间, 指令数↓ CPI↓ 主频↑
MIPS	$MIPS = \frac{IPS}{10^6} = \frac{f}{CPI \times 10^6}$	百万指令/秒	每秒执行百万条指令数, 不同架构不可直接比较
FLOPS	$FLOPS = \frac{\text{浮点操作数}}{\text{时间}}$	FLOPS	M FLOPS : 百万 10^6 G FLOPS : 十亿 10^9 T FLOPS : 万亿 10^{12} P FLOPS : 千万亿 10^{15} E FLOPS : 百京 10^{18} , 1京 = 1亿亿 = 10^{16} Z FLOPS : 十万京 10^{21}

字、字长、机器字长、指令字长、存储字长的区别与联系

概念	核心定义	决定方	说明
字	计算机一次并行处理的一组二进制位	系统架构设计	逻辑处理单位, 其长度即字长
字长	一个字包含的二进制位数	通常等同于机器字长	反映系统基本数据处理能力
机器字长	CPU 一次能处理的数据位数 (通用寄存器/ALU 宽度)	CPU 硬件设计	核心基准 , 决定数据处理能力、运算精度

概念	核心定义	决定方	说明
指令字长	一条机器指令的二进制编码位数	指令集架构 (ISA) 规范	可定长或变长, 影响取指效率
存储字长	一个内存单元存储的二进制位数 (一次访存取数宽度)	内存硬件设计 (如 DRAM 标准)	需匹配总线, 影响访存效率

例 1.1: 假定计算机 M1 和 M2 具有相同的指令集体系结构, M1 的主频为 $2GHz$, M1 上的运行时间为 $10s$ 。M2 采用新技术可使主频大幅提升, 但平均 CPI 也增加到 M1 的 1.5 倍。则 M2 的主频至少提升到多少才能使程序 P 在 M2 上的运行时间缩短为 $6s$?

解答

程序 P 在 M1 上的时钟周期数 = 指令条数 $\times$ 平均 $CPI = 10s \times 2GHz = 2 \times 10^{10}$

M2 的平均 CPI 为 M1 的 1.5 倍, 因此程序 P 在 M2 上的时钟周期数 = $1.5 \times 2 \times 10^{10} = 3 \times 10^{10}$

要使程序 P 在 M2 上的运行时间缩短到 $6s$, 则 M2 的主频至少应为

$$\frac{\text{程序 } P \text{ 在 } M2 \text{ 上的时钟周期数}}{\text{CPU 执行时间}} = \frac{3 \times 10^{10}}{6s} = 5GHz$$

由此可见, M2 的主频是 M1 的 2.5 倍, 但 M2 的速度却只是 M1 的 1.67 倍

第 2 章 数据的表示与计算

数制与编码

定义

r 进制数 $K_n K_{n-1} \dots K_0 K_{-1} \dots K_{-m}$ 的数值可表示为:

$$K_n r^n + K_{n-1} r^{n-1} + \dots + K_0 r^0 + K_{-1} r^{-1} + \dots + K_{-m} r^{-m} = \sum_{i=n}^{-m} K_i r^i$$

1. 基数: 每个数位所用到的不同数码的个数, r 进制基数为 r
2. 位权: r^i
3. 后缀字母表示: B (二进制)、O (八进制)、D (十进制)、H (十六进制)

进制转换

1. 二进制转十进制: 求加权和, 基数为 2
2. 八进制转十进制: 求加权和, 基数为 8
3. 十六进制转十进制: 求加权和, 基数为 16
4. 二进制转八进制: 3 位一转
5. 二进制转十六进制: 4 位一转
6. 八进制转二进制: 1 位转 3 位
7. 十六进制转二进制: 1 位转 4 位
8. 十进制转其他进制: 整数部分除基取余, 除到 0 为止; 小数部分乘基取整, 乘到 1 为止

【例 2.2】将十进制数 123.6875 转换成二进制数。

解：

除基取余法（整数部分）：整数部分除基取余，最先取得的余数为数的最低位，最后取得的余数为数的最高位（除基取余，先余为低，后余为高），商为 0 时结束。

整数部分：

除基	取余	
2 1 2 3	1	最低位
2 6 1	1	
2 3 0	0	
2 1 5	1	
2 7	1	
2 3	1	
2 1	1	最高位
0		

因此整数部分 $123 = (1111011)_2$ 。

乘基取整法（小数部分）：小数部分乘基取整，最先取得的整数为数的最高位，最后取得的整数为数的最低位（乘基取整，先整为高，后整为低），乘积为 1.0（或满足精度要求）时结束。

小数部分：

乘基	取整	
0.6875		
$\times 2$		
1.3750	1	最高位
0.3750		
$\times 2$		
0.7500	0	
$\times 2$		
1.5000	1	
0.5000		
$\times 2$		
1.0000	1	最低位
.....		

因此小数部分 $0.6875 = (0.1011)_2$ ，所以 $123.6875 = (1111011.1011)_2$ 。

注意

关于十进制数转换为任意进制数为何采用除基取余法和乘基取整法，以及所取之数放置位置的原理，请结合 r 进制数的数值表示公式思考，而不应死记硬背。

瞪眼法： $(11.625)_{10}$ 转二进制

$$11 = 8 + 2 + 1 = 2^3 + 2^1 + 2^0 = (1011)_2$$

$$0.625 = 0.5 + 0.125 = 2^{-1} + 2^{-3} = (0.101)_2$$

$$\text{于是 } (11.625)_{10} = (1011.101)_2$$

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

定点数

1. 真值与机器数：真值几进制都可但要带正负号，机器数二进制表示且正负号数字化
2. 定点数与浮点数：定点数小数点位置固定（定点整数、定点小数），浮点数小数点位置不固定
3. 定点整数

编码	方式	范围（字长为 n ）	特点	产生的问题
原码	最高位 1 表示负数，其余位表示数的绝对值	$[-(2^{n-1}-1), 2^{n-1}-1]$	正负数对称	1000 表示 -0

编码	方式	范围 (字长为 $n + 1$)	特点	产生的问题
反码	正数同原码; 负数为数值位按位取反	$[-(2^n - 1), 2^n - 1]$	原-反转换直接	1111 表示 -0
补码	正数同原码; 负数为数值位取反加 1	$[-2^n, 2^n - 1]$	加减运算统一, 零唯一	范围不对称
移码	补码符号位取反, $[x]_{\text{移}} = 2^n + x \quad (-2^n \leq x \leq 2^n)$	$[-2^n, 2^n - 1]$	便于浮点数阶码比较大小	

- 补码的取模意义: 补码是一种模运算系统, 利用固定模数 2^n 将负数映射到正数域
运算同一性: 加减法均转化为模 2^n 加法, 硬件实现无需区分符号
- 补码表示的最小值, 原反码表示不了, 需要单独记忆
- 补码转原码
 - a. $[[x]_{\text{补}}]_{\text{补}} = [x]_{\text{原}}$
 - b. 先末尾减一, 再符号位不变, 数值位取反
- 求 $[-x]_{\text{补}}$: 对 $[x]_{\text{补}}$ 的各个位 (包括符号位) 按位取反, 末尾加一
- 机器数与真值的单调性比较

编码	正半轴单调性	负半轴单调性	全局单调性
原码	✓ 一致	✗ 相反	✗ 不一致
反码	✓ 一致	✓ 一致	✗ 不一致*
补码	✓ 一致	✓ 一致	✓ 一致**
移码	-	-	✓ 一致

*注: 反码在正负零处不连续 (0000 (+0) 和 1111 (-0)), 但单调性在负半轴仍一致

**注: 补码在 0 处连续 (0000 $\rightarrow$ 1111), 真值 $-1 \rightarrow 0 \rightarrow +1$ 对应机器数递增

- 无符号数: 当一个编码的所有二进制位都用来表示数值而没有符号位时, 该编码表示的就是无符号数

4. 定点小数: 原码小数, 用在 IEEE754, 小数点在符号位后面, 固定

C 语言中的整数类型及类型转换

1. char: 有的编译器定义为有符号数, 有的定义为无符号数
2. 类型转换: 强制类型转换并不会改变变量的机器数, 只是改变机器数的解释方法
3. 不同字长整数之间的转换
 - 长变短: 高位截断, 低位保留
 - 短变长: 高位扩展, 低位复制 (使用原数字的 **符号位** 扩展高位)

运算方法和运算电路

补码加减运算

```
1  /**
2   * @brief 计算整数的相反数
3   * @param x 一个整数
4   * @return x的相反数
5   * @example
6   * assert(neg(-1) == 1);
7   * assert(neg(1) == -1);
8   * assert(neg(100) == -100);
9   * assert(neg(-100) == 100);
10  */
11 int neg(int x) {
12     return ~x + 1;
13 }
```

补码的溢出判断

1. 若 **正数 + 正数 = 负数** 或 **负数 + 负数 = 正数**, 则溢出; 否则未溢出
2. 若最高位进位与次高位进位若不同, 则溢出; 否则未溢出 ($OF = C_1 \oplus C_2$)
3. 使用双符号位计算, 计算结果符号位的各种情况如下
 - $K_1K_2 = 00$: 表示结果是正数, 无溢出
 - $K_1K_2 = 01$: 表示结果正溢出 (比最大值还大)
 - $K_1K_2 = 10$: 表示结果负溢出 (比最小值还小)
 - $K_1K_2 = 11$: 表示结果是负数, 无溢出

而模4补码, 就是将模长变为4, 此时, 正数部分, $[x]_{补} = (x+4)\%4$, 负数部分, $[x]_{补} = (x+4)\%4$ 。

例如,

$x=0.125$ 时, $[x]_{补} = (4+0.125)\%4 = 00.125 = 00.001B$

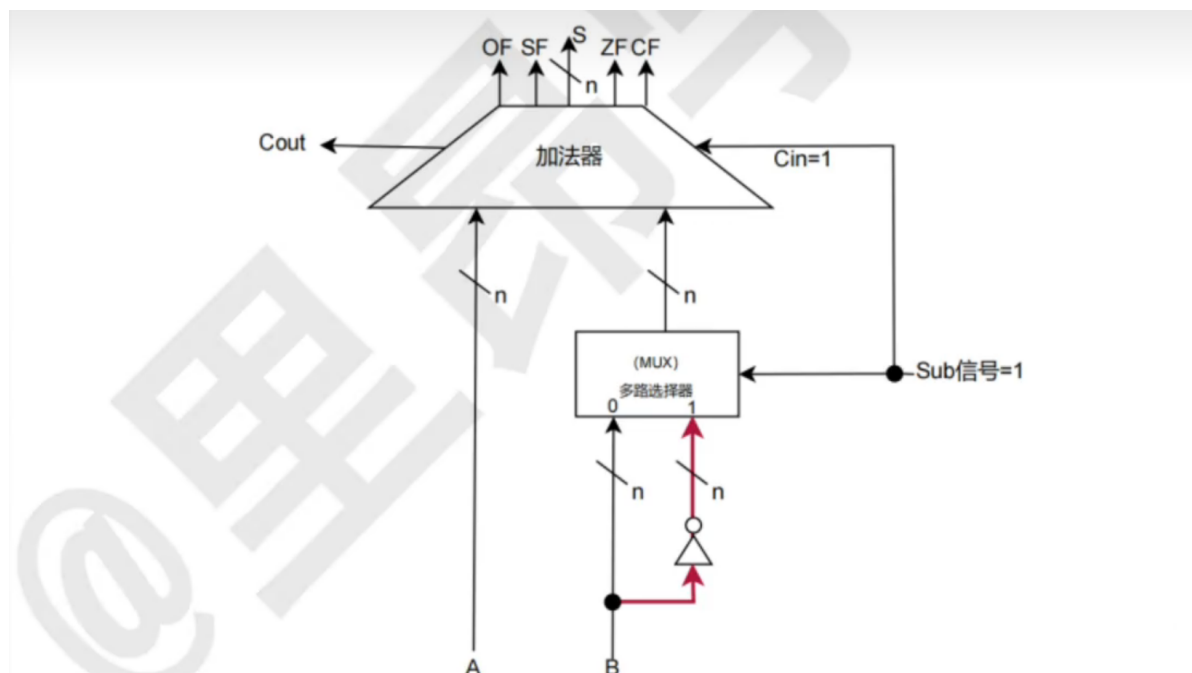
$x=-1$ 时, $[x]_{补} = [4+(-1)]\%4 = 3$, 即11.000B

$x=-0.125$ 时, $[x]_{补} = [4+(-0.125)]\%4 = 3.875 = 11.111B$

此时, **符号位就由单符号位变成了双符号位**, 同理, 对于定点正数, 将模长变为原来的两倍, 则也可以将单符号位变成双符号位, 这里不做赘述。

任何一个正确的数值, 模4补码的两个符号位都是相同的, 所以只需要存一位即可

加减运算电路



若Sub=1，则表示做减法操作，选择多路选择器的第二路信号作为输出，即让~B通过，此时C_{in}=1。

不难发现，这里其实是对B做了变补操作，即（包含符号位）按位取反，末位加1。本质上是利用了补码减法运算规则：

$$[A-B]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} \pmod{2^{n+1}}$$

接着加法器会做加法操作，计算 $A + (-B) + C_{in}$ ，并更新OF, SF, ZF, CF等标志位。（我们一会详细讨论这些标志位）

- 减法标志 Sub : $Sub = 0$ 减法, $Sub = 1$ 加法
- $C_{in} = Sub$
- $C_{out} = C_n$, 第 n 位进位, 即最高位的进位输出
- S : 运算结果, n 位
- 零标志 ZF: $ZF = 1$ 结果 F 为 0, $ZF = 0$ 结果 S 不为 0
- 溢出标志 OF: $OF = C_n \oplus C_{n-1}$, $OF = 1$ 有符号数运算溢出, $OF = 0$ 有符号数运算未溢出。对无符号数运算无意义
- 符号标志 SF: $SF = S$ 的最高位, $SF = 1$ 结果为负, $SF = 0$ 结果为正
- 进/借位标志 CF: $CF = Sub \oplus C_{out}$ ($C_{in} = Sub$), $CF = 1$ 无符号数运算溢出, $CF = 0$ 无符号数运算未溢出。对有符号数运算无意义
 - $CF = 1$: 无符号数加法最高位产生进位 或 无符号数减法最高位产生借位

无符号数 大小的比较

1. $ZF = 1$, 说明 $A = B$
2. $ZF = 0$ 且 $CF = 0$, 说明 $A > B$
3. $CF = 1$, 说明 $A < B$

有符号数 大小的比较

1. $ZF = 1$, 说明 $A = B$
2. $ZF = 0$ 且 $OF = SF$ (或 $OF \oplus SF = 0$), 说明 $A > B$
3. $ZF = 0$ 且 $OF \neq SF$ (或 $OF \oplus SF = 1$), 说明 $A < B$

算加减法判断溢出一定要看 OF/CF, 最高位超出字长不一定代表着溢出, 例如 8 位字长下 11101000 和 11011000 相加得到 11000000

标志寄存器（程序状态寄存器 PSW）

- 属于运算器
- 基于 x86 架构 EFLAGS 寄存器的标准布局：

位数	标志位	说明
0	CF	进位标志
1		(保留位, 通常为 1)
2	PF	奇偶标志
3		(保留位, 通常为 0)
4	AF	辅助进位标志
5		(保留位, 通常为 0)
6	ZF	零标志
7	SF	符号标志
8	TF	陷阱标志
9	IF	中断允许标志
10	DF	方向标志
11	OF	溢出标志
12	IOPL0	I/O 特权级别低位
13	IOPL1	I/O 特权级别高位
14	NT	嵌套任务标志
15		(保留位, 通常为 0)

移位运算与乘除法运算

1. 逻辑移位：不考虑符号位

- 左移时，高位移出，低位补 0
- 右移时，低位移出，高位补 0

算术移位（采用袁春风老师的解释方式）

- 左移时，高位移出，低位补 0
- 右移时，低位移出，高位补符号位

2. 无符号乘法的溢出判断：两个 n 位无符号数相乘，结果为 $2n$ 位，若高 n 位不全为 0，说明发生溢出，将 OF 设为 1

有符号乘法的溢出判断：两个 n 位有符号数相乘，结果为 $2n$ 位，若高 $n + 1$ 位不完全相同，说明发生溢出，将 OF 设为 1

3. Booth 乘法：补码乘法算法

4. 阵列乘法器：快速乘法器的一种，一个时钟周期内计算 n 位乘法

5. 无符号整数除法：TODO

浮点数的表示和运算

1. 上溢会报错中断，下溢直接变成 0 不中断

2. float: 1-8-23

double: 1-11-52

阶码移位偏置值为 $2^{n-1} - 1$

◦ float: $2^{8-1} - 1 = 127$

◦ double: $2^{11-1} - 1 = 1023$

3.

第 3 章 存储系统

第 4 章 指令系统

第 5 章 中央处理器

第 6 章 总线

第 7 章 输入/输出系统

综合题

Cache

虚拟存储器

数据寻址与指令寻址

高级语言与机器语言的对应

数据通路的功能结构

指令流水线